
Profesjonalny skład tek- stu bez tajemnic

2026

Spis treści

1. Dlaczego LaTeX, a nie Word? Filozofia narzędzia, które myśli za Ciebie	5
1.1. Od Knutha do Lamporta: 40 lat typografii, której nie widać	5
1.2. WYSIWYG kontra WYSIWYM: Dwa różne kontrakty z autorem	7
1.3. Kiedy $\LaTeX$ wygrywa, kiedy przegrywa: Uczciwa mapa zastosowań	9
1.4. Ekosystem narzędzi: $\TeX$ Live, MiKTeX i Overleaf — którą drogę wybrać na start	12
2. Pierwszy dokument w 30 minut: Struktura, klasy i pakiety, które musisz znać	17
2.1. Anatomia pliku <code>.tex</code> : Preambuła, ciało, kompilacja — bez czarnej magii	18
2.1.1. Gdy kompilacja się nie udaje: Czytanie błędów i skuteczne debugowanie	19
2.2. Klasy dokumentów i opcje: <code>article</code> , <code>report</code> , <code>book</code> — i kiedy użyć każdej	21

2.3. Pakiety, które zmieniają wszystko: geometry, fancyhdr, graphicx i babel po polsku	24
2.4. Znaki specjalne, cudzysłowy i polskie ogonki: Pułapki, na które wpada każdy początkujący .	26
3. Tekst, który wygląda profesjonalnie: Formatowanie, listy, tabele i grafika	31
3.1. Hierarchia tekstu: Sekcje, podsekcje, spis treści i automatyczna numeracja	31
3.2. Wyróżnienia, listy i środowiska: itemize, enumerate, description i bloki cytatów	33
3.3. Tabele bez bólu głowy: Środowisko tabular, pozycjonowanie i pakiet booktabs	36
3.4. Grafika i wstawki: includegraphics, figure, pozycjonowanie i podpisy	38
3.5. Matematyka w praktyce: Od wzoru inline do równań numerowanych z pakietem amsmath . . .	40
4. Od pliku do gotowej pracy: Bibliografia, skorowidze i własne szablony	45
4.1. Bibliografia bez ręcznego formatowania: BibTeX, plik .bib i style cytowań	45
4.1.1. Biber i BibLaTeX: nowoczesna alternatywa, której wymaga coraz więcej uczelni . . .	49
4.1.2. Zarządzanie dużymi projektami: \input, \include i automatyzacja z latexmk . . .	50
4.2. Odsyłacze, skorowidze i przypisy: \label, \ref, \footnote i makeindex	52

4.3. Własne komendy i pakiety: Jak przestać kopio- wać kod i zacząć myśleć jak programista	54
4.4. Co dalej? Mapa zasobów: CTAN, GUST, Overleaf Learn i TeX Stack Exchange	56
4.5. Dokąd stąd: Podsumowanie całej drogi	58

Rozdział 1.

Dlaczego LaTeX, a nie Word? Filozofia narzędzia, które myśli za Ciebie

Większość ludzi zaczyna pisać dokumenty tak, jak się je zawsze pisało: otwierasz edytor, klikasz, widzisz wynik. Brzmi rozsądnie. Ale gdy piszesz pracę dyplomową liczącą 80 stron, ze wzorami matematycznymi, automatyczną bibliografią i numerowanymi rozdziałami, ta prostota zaczyna cię kosztować coraz więcej czasu. $\LaTeX$ rozwiązuje ten problem inaczej — nie pytając cię, jak coś ma wyglądać, lecz skupiając się na tym, czym coś *jest*.

1.1 Od Knutha do Lamporta: 40 lat typografii, której nie widać

Donald Knuth — matematyk, informatyk i autor wielotomowego *The Art of Computer Programming* — w 1974 roku przestał wysyłać swoje prace do American Mathematical Society.

Powód był prozaiczny: wydrukowane artykuły wyglądały po prostu brzydko. Komputerowy skład tamtej epoki nie dorównywał dziełom ręcznych zecerów, a Knuth nie mógł znieść widoku własnych wzorów matematycznych złożonych niedbale. W 1977 roku zabrał się do pisania własnego systemu składu. Efekt — $\text{\TeX}$ — udostępnił publicznie w 1982 roku.

$\text{\TeX}$ jest programem niezwykle stabilnym. Knuth celowo zamroził jego specyfikację, a wersje numeruje zbieżnymi do liczby π kolejnymi przybliżeniami: 3, 3.1, 3.14, 3.141 i tak dalej. Obecna wersja to 3.141592653. Każda zmiana jest poprawką błędu, nigdy nową funkcją. Dla użytkownika oznacza to, że dokument napisany w 1985 roku skompiluje się dziś bez modyfikacji — gwarancja nieosiągalna dla żadnego formatu `.docx`.

Leslie Lamport stworzył $\text{\LaTeX}$ w latach 80. jako zestaw makr i poleceń wysokiego poziomu zbudowanych na $\text{\TeX}$ -ie. Zamiast ręcznie sterować każdym elementem składu, autor mógł pisać `\section{Wstęp}` i zapomnieć o tym, jak sekcja ma wyglądać — system sam o to dbał. Pierwsza wersja $\text{\LaTeX}$ -a 2.09 szybko zyskała popularność w środowisku akademickim i naukowym, ale przez lata powstawały niekompatybilne rozszerzenia i odgałęzienia.

Odpowiedzią była $\text{\LaTeX}2_{\epsilon}$, wydana w 1994 roku przez tak zwany *LaTeX3 team* pod kierownictwem Franka Mittelbacha. Ujednoliciła ona rozbite warianty w jeden spójny system i do dziś pozostaje standardem. Drużyna $\text{\LaTeX}3$ pracuje równolegle

nad kolejną generacją — jednak świadomie nie śpieszy się z wdrożeniem, bo stabilność traktuje jako wartość, nie jako stagnację.

Stabilność $\text{\TeX}$ -a to celowy projekt. Numery wersji $\text{\TeX}$ -a zbiegają do π — Knuth zaprojektował system tak, by nie zmieniał się bez powodu. Dokument napisany 40 lat temu kompiluje się dziś identycznie. Żaden format binarny, w tym `.docx`, nie oferuje takiej gwarancji.

1.2 WYSIWYG kontra WYSIWYM: Dwa różne kontrakty z autorem

Microsoft Word działa na zasadzie *What You See Is What You Get* — widzisz sformatowany tekst i bezpośrednio na nim pracujesz. $\text{\LaTeX}$ realizuje koncepcję *What You See Is What You Mean*: piszesz kod opisujący strukturę i znaczenie dokumentu, a system sam przekłada go na skład.

Różnica staje się widoczna przy konkretnych zadaniach. Numeracja rozdziałów w Wordzie wymaga list wielopoziomowych, odpowiednio skonfigurowanych stylów i — jeśli coś pójdzie nie tak — kilkunastu minut klikania. W $\text{\LaTeX}$ -u wystarczy `\chapter{}` i `\section{}`: numery aktualizują się automatycznie, zawsze i wszędzie, łącznie ze spisem treści generowanym poleceniem `\tableofcontents`.

Wzory matematyczne to różnica jeszcze bardziej rażąca. Word

oferuje edytor równań, który przy złożonych wyrażeniach staje się nieintuicyjny. $\LaTeX$ zapisze całkę oznaczoną jako $\int_a^b f(x)dx$ w kilka sekund, a wynik typograficznie przewyższy wszystko, co potrafi wygenerować edytor graficzny.

Tabela 1.1: Porównanie $\LaTeX$ -a i Worda w typowych zadaniach akademickich

Zadanie		
Numeracja rozdziałów	Automatyczna, bez konfiguracji	Wymaga stylów i list wielopoziomowych
Wzory matematyczne	Precyzyjny skład, pełna kontrola	Edytor graficzny, ograniczona złożoność
Bibliografia	BibTeX — jedno polecenie	Ręczna lub wtyczki zewnętrzne
Spis treści	Generowany z kodu źródłowego	Automatyczny, lecz podatny na błędy
Długie dokumenty	Stabilny przy setkach stron	Spowalnia i zawiesza się
Krzywa uczenia się	Stroma na początku	Natychmiastowa dla prostych zadań

Słabością $\LaTeX$ -a jest to samo, co jego siłą: dystans między pisanem a widzeniem efektu. Autor pisze kod, kompiluje go i

dopiero wtedy ogląda PDF. Jeśli potrzebujesz szybkiej notatki na spotkanie lub jednorazowego pisma do urzędu, ten cykl jest po prostu zbędny.

Nie rób z $\LaTeX$ -a narzędzia do wszystkiego $\LaTeX$ nie zastępuje Worda w każdej sytuacji. Dla dokumentów biurowych, szybkich notatek i korespondencji Word jest po prostu szybszy. Używaj $\LaTeX$ -a tam, gdzie liczy się precyzja struktury i powtarzalność formatowania — nie wszędzie.

WYSIWYM oznacza, że $\LaTeX$ zna kontekst. Gdy piszesz `\section{Metodologia}`, $\LaTeX$ wie, że to sekcja — i sam decyduje, jak ją sformatować zgodnie z klasą dokumentu. Word widzi tylko tekst z przypisanym stylem, który możesz przypadkowo nadpisać.

1.3 Kiedy $\LaTeX$ wygrywa, kiedy przegrywa: Uczciwa mapa zastosowań

Wyobraź sobie studenta czwartego roku informatyki piszącego pracę inżynierską. Ma 60 stron, 30 wzorów, 3 algorytmy, bibliografię liczącą 45 pozycji i rysunek w każdym rozdziale. W Wordzie spędził dwa wieczory tylko na tym, żeby numeracja rysunków zgadzała się ze spisem treści po dodaniu nowego rozdziału. W $\LaTeX$ -u środowisko `figure` z poleceniem `\label` i `\ref` rozwiązuje ten problem raz na zawsze.

Praca dyplomowa to klasyczny przypadek, gdzie $\LaTeX$ błyszczy.

Uczelnie coraz częściej udostępniają gotowe szablony `.tex` dostosowane do wymogów formatowania; student dostaje strukturę i skupia się na treści. Wydział Matematyki i Informatyki Uniwersytetu Adama Mickiewicza w Poznaniu czy Politechnika Warszawska publikują takie szablony oficjalnie. Overleaf — platforma online omówiona w kolejnym podrozdziale — hostuje setki szablonów dyplomowych polskich uczelni.

Artykuły naukowe to drugi naturalny teren $\LaTeX$ -a. Praktycznie każde wydawnictwo z dziedziny matematyki, fizyki, informatyki i inżynierii wymaga pliku `.tex` przy zgłaszaniu manuskryptu. Elsevier, Springer i IEEE dostarczają własne klasy dokumentów (`.cls`), które formatują artykuł zgodnie ze standardem danego czasopisma automatycznie — autor zmienia klasę i gotowe.

Książki techniczne i dokumentacja to kolejna mocna strona systemu. Pakiet `hyperref` generuje klikalny PDF z zakładkami i linkami, `makeindex` tworzy szczegółowy skorowidz, a podział na pliki przez `\include` sprawia, że projekt liczący 500 stron zarządza się tak samo sprawnie jak 50-stronicowy artykuł.

Przypadek: praca dyplomowa w 3 tygodnie zamiast 3 miesięcy
Studentka bioinformatyki na Uniwersytecie Warszawskim opisała publicznie na forum GUST, że pierwsze 40 stron pracy dyplomowej pisała w Wordzie przez sześć tygodni. Po przejściu na $\LaTeX$ z gotowym szablonem uczelnianym — kolejne 60 stron zajęło jej trzy tygodnie, mimo że materiał był trudniejszy.

Kluczowa różnica: nie traciła czasu na walkę z formatowaniem po każdej większej zmianie struktury.

Gdzie $\LaTeX$ przegrywa? Scenariusz pierwszy: raport z wyników kwartalnych dla działu finansów, z logotypem firmy, kolorowymi tabelami i wykresem z Excela. Tu Word albo Google Docs będą szybsze o rząd wielkości. Scenariusz drugi: dokumentacja projektowa tworzona przez cztery osoby bez dostępu do Overleaf, z komentarzami inline i śledzeniem zmian. Tryb recenzji Worda jest tu bezkonkurencyjny.

Zasada praktyczna: jeśli dokument zmienia się raz i idzie do druku lub na arXiv — $\LaTeX$ jest lepszym wyborem. Jeśli dokument jest żywym bytem, edytowanym przez wiele osób bez technicznego przygotowania — lepiej sięgnąć po inne narzędzie.

Kryterium wyboru jest prostsze, niż myślisz $\LaTeX$ opłaca się wszędzie tam, gdzie struktura dokumentu jest złożona i powtarzalna: wzory, bibliografia, numeracja rozdziałów, odsyłacze. Gdy dokument jest prosty i jednorazowy, dodatkowa inwestycja w naukę składni nie zwraca się.

1.4 Ekosystem narzędzi: $\TeX$ Live, $\text{MiK}\TeX$ i Overleaf — którą drogę wybrać na start

Zanim napiszesz pierwszą linię kodu, potrzebujesz *kompi-latora* — programu, który przetworzy plik `.tex` na PDF. Na rynku dominują trzy rozwiązania, które różnią się istotnie pod względem instalacji, rozmiaru i przeznaczenia.

T_EX Live to dystrybucja rozwijana przez T_EX Users Group i dostępna na Windows, macOS oraz Linux. Pełna instalacja zajmuje około 7 GB i zawiera praktycznie wszystkie pakiety dostępne w archiwum CTAN. Aktualizacje wychodzą raz do roku jako nowe wydanie; między wydaniem można aktualizować pakiety ręcznie przez menedżer `tlmgr`. Na Linuksie T_EX Live jest domyślnym wyborem — instalacja przez menedżer pakietów (`apt install texlive-full`) zajmuje minuty.

MiK_TE_X to dystrybucja pierwotnie przeznaczona dla Windows, choć od kilku lat dostępna też na macOS i Linux. Jej wyróżnikiem jest instalacja pakietów *na żądanie*: jeśli kompilowany dokument wymaga pakietu, którego jeszcze nie ma na dysku, MiK_TE_X pobiera go automatycznie podczas kompilacji. Instalacja startowa zajmuje poniżej 500 MB. Dla użytkownika Windows, który dopiero zaczyna i nie chce czekać na pobranie 7 GB, to rozsądny punkt wejścia.

Overleaf jest platformą działającą w przeglądarce: zero instalacji, zero konfiguracji. Wystarczy założyć konto i zacząć pisać. Darmowy plan pozwala kompilować dokumenty i zapraszać do projektu jedną osobę. Plan *Student* kosztuje około 15 USD miesięcznie i odblokowuje współpracę w czasie rzeczywistym

dla dowolnej liczby osób. Wiele uczelni — w tym Politechnika Gdańska — wykupuje licencje instytucjonalne, dzięki którym studenci korzystają z pełnych możliwości Overleaf za darmo.

Tabela 1.2: Porównanie środowisk L^AT_EX-a według kluczowych kryteriów

Kryterium			
System operacyjny	Linux, macOS, Windows	Windows (głównie)	Przeglądarka
Rozmiar instalacji	~7 GB	<500 MB	Brak instalacji
Pakiety	Wszystkie od razu	Pobierane na żądanie	Zarządzane przez platformę
Współpraca	Brak wbudowanej	Brak wbudowanej	Tak, w czasie rzeczywistym
Praca offline	Tak	Tak	Nie
Cena	Bezpłatny	Bezpłatny	Bezpłatny / od 15 USD/mies.

Dla studenta zaczynającego pierwszą pracę dyplomową rekomendacja jest jednoznaczna: zacznij od Overleaf. Nie tracisz czasu na konfigurację, masz dostęp do setek szablonów uczel-

nianych i możesz natychmiast pokazać plik promotorowi bez wysyłania załączników e-mailem. Gdy poczujesz się pewnie w składni $\text{\LaTeX}$ -a — a zajmie to kilka tygodni regularnego pisanie — migracja do lokalnej instalacji jest trywialna, bo plik `.tex` działa identycznie wszędzie.

Dla zawodowca, który pisze regularnie i potrzebuje pracy offline — na przykład w pociągu czy bez stabilnego łącza — $\text{\TeX}$ Live na macOS lub Linuksie jest właściwym wyborem. Pełna instalacja gwarantuje dostęp do każdego pakietu z CTAN bez niespodzianek podczas kompilacji. Na Windows MiKTeX sprawdza się równie dobrze, a automatyczne pobieranie pakietów eliminuje konieczność ręcznego zarządzania dystrybucją.

Pierwsze kroki z Overleaf w 5 minut Wejdź na overleaf.com, załóż bezpłatne konto i wybierz szablon *Blank Project*. Overleaf skompiluje minimalny dokument natychmiast. Zmień tekst między `\begin{document}` a `\end{document}`, kliknij *Recompile* i obserwuj wynik po prawej stronie. To wystarczy, by zrozumieć cykl pracy z $\text{\LaTeX}$ -em zanim zainstalujesz cokolwiek lokalnie.

Edytory lokalne warte uwagi to TeXstudio — otwartoźródłowy, z podpowiedziami składni i wbudowaną przeglądarką PDF — oraz Visual Studio Code z rozszerzeniem *LaTeX Workshop*, który sprawdzi się jeśli już używasz VS Code do innych projektów. Oba integrują się zarówno z $\text{\TeX}$ Live, jak i MiKTeX -em bez dodatkowej konfiguracji.

Wybór narzędzia nie powinien blokować nauki Overleaf dla studenta, T_EX Live dla zaawansowanego użytkownika pracującego offline — to dobre punkty startowe. Plik `.tex` jest zwykłym tekstem i działa identycznie w każdym środowisku, więc zmiana narzędzia w połowie projektu nie kosztuje nic.

Rozdział 2.

Pierwszy dokument w 30 minut: Struktura, klasy i pakiety, które musisz znać

Masz już wybrane narzędzie — Overleaf, TeXstudio albo lokalną instalację T_EX Live. Teraz czas napisać coś, co naprawdę się skompiluje. Ten rozdział przeprowadzi cię przez anatomię pliku `.tex` krok po kroku, pokaże, czym różni się `article` od `book` i wyjaśni, które pakiety musisz załadować, zanim zaczniesz pisać po polsku. Żadnej magii — tylko logika systemu, który zaprojektowano tak, żeby dało się go zrozumieć.

2.1 Anatomia pliku `.tex`: Preambuła, ciało, kompilacja — bez czarnej magii

Każdy plik `.tex` — bez wyjątku — dzieli się na dwie strefy. Pierwsza to *preambuła*, która zaczyna się od wiersza pierwszego i kończy tuż przed `\begin{document}`. Druga to *ciało*, zamknięte między `\begin{document}` a `.` Co znajdzie się za ostatnim poleceniem, $\LaTeX$ po prostu zignoruje.

Minimalny dokument, który skompiluje się bez błędu, wygląda tak:

```
" \documentclass{article}
  \begin{document}
  Witaj, świecie!
  \end{document}
```

Preambuła w tym przykładzie zawiera dokładnie jedno polecenie: `\documentclass{article}`. To wystarczy, bo $\LaTeX$ zna rozsądne wartości domyślne dla większości parametrów. W praktycznych dokumentach preambuła rozrasta się do kilkunastu wierszy — ładujesz w niej pakiety, definiujesz metadane i ustawiasz marginesy. Ciało zawiera to, co trafi na stronę.

Gdy klikasz *Compile* w Overleaf albo wpisujesz `pdflatex dokument.tex` w terminalu, uruchamia się trzystopniowy proces. `pdflatex` czyta plik `.tex` od góry do dołu, przetwarza polecenia i generuje trzy pliki wyjściowe: gotowy `.pdf`, plik pomocniczy `.aux` z informacjami o odsyłaczach i numerach stron, oraz `.log` z pełnym dziennikiem kompilacji. Plik `.aux` jest potrzebny przy kolejnym przebiegu — dlatego dokumenty ze spisem treści lub odsyłaczami wymagają zwykle dwóch kompilacji z rzędu.

Plik .log to twoje pierwsze źródło diagnozy błędów. Overleaf wyświetla go bezpośrednio w zakładce *Logs & output files*. Gdy kompilacja kończy się komunikatem ! Undefined control sequence, oznacza to, że użyłeś polecenia, którego $\LaTeX$ nie zna — najczęściej dlatego, że brakuje pakietu w preambule. Komunikat Runaway argument pojawia się natomiast wtedy, gdy zapomnisz zamknąć nawias klamrowy. Oba błędy zobaczysz dziesiątki razy na początku nauki i oba naprawiasz w mniej niż minutę.

Kompiluj po każdej zmianie Nie pisz dwóch stron i dopiero wtedy kompiluj. Uruchamiaj pdflatex po każdym akapicie. Krótki cykl kompilacji skraca czas lokalizacji błędu z godziny do kilku sekund — błąd, który popełniłeś w ostatnich trzech wierszach, znajdziesz natychmiast.

Dwa pliki, które musisz rozumieć .aux przechowuje numery stron i odsyłacze między przebiegami kompilacji; .log rejestruje każdą operację i każdy błąd. Gdy coś nie działa, otwierasz .log i szukasz wiersza zaczynającego się od wykrzyknika. Reszta pliku to tylko kontekst.

2.1.1 *Gdy kompilacja się nie udaje: Czytanie błędów i skuteczne debugowanie*

Każdy użytkownik $\LaTeX$ prędzej czy później zobaczy w terminalu czerwony komunikat i pytanie ?. Kluczem do spokoju jest umiejętność **czytania logu**, a nie panikowanie.

Plik `.log` powstaje przy każdej kompilacji i zawiera pełną historię przetwarzania dokumentu. Błędy zaczynają się od wykrzyknika, np.:

```
! Undefined control sequence.
```

```
1.23 \tytul
```

```
    {Mój dokument}
```

Ten komunikat mówi nam dokładnie: *nieznana komenda* `\tytul` w **linii 23**. Wystarczy sprawdzić pisownię — poprawna komenda to `\title`.

Najczęstsze błędy i ich przyczyny:

- `! Missing $ inserted` — użyłeś symbolu matematycznego (np. podkreślenia `_`) poza środowiskiem matematycznym.
- `! File 'pakiet.sty' not found` — pakiet nie jest zainstalowany; w TeX Live uruchom `tlmgr install pakiet`.
- `! Runaway argument` — brakuje zamykającego nawiasu klamrowego `}` w jednym z poleceń.
- `Overfull \hbox` — to **ostrzeżenie**, nie błąd; $\TeX$ nie mógł złamać wiersza elegancko, ale dokument się skompiluje.

Strategia binarnego szukania błędu Jeśli nie możesz zlokalizować błędu, zastosuj metodę połowienia: zakomentuj (%) połowę treści dokumentu i skompiluj ponownie. Jeśli błąd zniknie, problem leży w zakomentowanej części. Powtarzaj, aż znajdziesz winną linię.

Nie ignoruj numeru linii Komunikat 1.23 wskazuje linię, w której $\LaTeX$ zorientował się o błędzie — rzeczywista przyczyna może leżeć kilka linii wcześniej, np. w niezamkniętym środowisku `\begin{itemize}`.

W edytorach takich jak TeXstudio czy Overleaf błędy są podświetlane bezpośrednio w kodzie, co znacznie przyspiesza pracę. Mimo to warto raz przejrzeć surowy plik `.log` — zawiera informacje, których interfejs graficzny nie wyświetla.

2.2 Klasy dokumentów i opcje: `article`, `report`, `book` — i kiedy użyć każdej

Wybór klasy dokumentu to pierwsza decyzja architektoniczna, którą podejmujesz w pliku `.tex`, i jedyna, której zmiana w połowie projektu może wymagać korekty kilku poleceń. Trzy standardowe klasy — `article`, `report` i `book` — różnią się nie tylko wyglądem, lecz przede wszystkim tym, jakie polecenia strukturalne udostępniają.

`article` nie ma rozdziałów. Najwyższy poziom hierarchii to `\section{}`, poniżej `\subsection{}` i `\subsubsection{}`. Klasa ta nie generuje strony tytułowej automatycznie ani nie zaczyna nowej strony po tytule. Sprawdza się idealnie dla artykułów naukowych, dokumentacji technicznej i krótkich opracowań do kilkudziesięciu stron.

`report` dodaje `\chapter{}` jako najwyższy poziom i domyśl-

nie tworzy stronę tytułową po wywołaniu `\maketitle`. Każdy rozdział zaczyna się od nowej strony. To naturalna klasa dla prac dyplomowych, raportów semestralnych i dłuższych dokumentów technicznych. Domyślna opcja łamania rozdziałów to `openany` — rozdział może zaczynać się na parzystej lub nieparzystej stronie.

`book` to klasa dla dokumentów przeznaczonych do druku. Domyślnie włącza skład dwustronny (`twoside`) i ustawia opcję `openright`, przez co każdy rozdział zaczyna się na stronie nieparzystej. Numeracja stron pojawia się naprzemiennie na lewym i prawym marginesie, co odpowiada konwencji książkowej. Klasa udostępnia też polecenia `\frontmatter`, `\mainmatter` i `\backmatter` do organizacji przedniej i tylnej materii.

Tabela 2.1: Porównanie standardowych klas dokumentów $\LaTeX$

Cecha			
Najwyższy poziom	<code>\section</code>	<code>\chapter</code>	<code>\chapter</code>
Strona tytułowa	opcjonalna	domyślna	domyślna
Skład dwustronny	nie	nie	tak
Numeracja stron	dół, środek	górze, wewnątrz	naprzemiennie
Typowe zastosowanie	artykuły, raporty	prace dyplomowe	książki

Opcje przekazujesz w nawiasach kwadratowych, oddzielając je przecinkami:

```
“ \documentclass[12pt, a4paper, twoside]{report}
```

Opcja `12pt` zwiększa domyślny rozmiar pisma z 10 do 12 punktów — wszystkie inne elementy (tytuły sekcji, przypisy) skalują się proporcjonalnie. `a4paper` ustawia format papieru A4, bo bez tej opcji $\LaTeX$ przyjmuje `letterpaper` zgodnie z amerykańską konwencją. `twoside` aktywuje skład dwustronny z asymetrycznymi marginesami nawet w klasie `report`. Dla pracy dyplomowej drukowanej dwustronnie to ustawienie obowiązkowe.

Nie zmieniaj klasy w połowie projektu. Przejście z `article` na `report` po napisaniu trzydziestu stron wymaga zamiany wszystkich `\section` na `\chapter` i dostosowania hierarchii nagłówków. Wybierz klasę przed pierwszym akapitem tekstu i trzymaj się jej.

Klasa to architektura dokumentu `article` — krótkie teksty bez rozdziałów. `report` — prace dyplomowe i wielostronicowe raporty. `book` — dokumenty przeznaczone do druku z przednią i tylną materią. Ta decyzja determinuje strukturę hierarchii i domyślny wygląd całości.

2.3 Pakiety, które zmieniają wszystko: geometry, fancyhdr, graphicx i babel po polsku

Standardowy $\text{\LaTeX}$ ma rozsądne wartości domyślne, ale do profesjonalnego dokumentu po polsku potrzebujesz czterech pakietów, które rozwiązują cztery konkretne problemy: marginesy, nagłówki, grafikę i polskie znaki.

`geometry` to najprostszy sposób na ustawienie marginesów. Bez niego $\text{\LaTeX}$ stosuje własne obliczenia bazujące na klasie i rozmiarze pisma, które dla formatu A4 dają marginesy znacznie szersze niż wymaga większość uczelni. Polecenie `ustawia` wszystkie cztery marginesy na 2,5 cm jedną opcją. Dla pracy dyplomowej z bindownicą: `[left=3.5cm, right=2.5cm, top=2.5cm, bottom=2.5cm]`.

`fancyhdr` daje kontrolę nad nagłówkami i stopkami. Domyślny styl `headings` wyświetla tytuł sekcji i numer strony, ale wygląd jest surowy. Po załadowaniu pakietu wywołujesz `\pagestyle{fancy}` i definiujesz trzy pola nagłówka poleceniami `\lhead{}`, `\chead{}`, `\rhead{}` oraz analogicznie dla stopki. Dla pracy dyplomowej typowy układ to tytuł rozdziału po lewej i numer strony po prawej.

`graphicx` obsługuje wstawianie grafiki. Po załadowaniu pakietu polecenie `\includegraphics[width=0.8\textwidth]{rysunek.pdf}` wstawia plik graficzny i skaluje go do 80% szerokości kolumny tekstu. Pakiet rozumie formaty PDF, PNG i JPEG przy kompilacji przez `pdflatex`. Stary format EPS wymaga konwersji

do PDF lub użycia kompilatora latex (który generuje DVI, nie PDF).

Polskie znaki diakrytyczne to osobny temat z kilkoma ważnymi decyzjami. Klasyczne podejście łączy dwa pakiety:

```
“ \usepackage[T1]{fontenc}
  \usepackage[utf8]{inputenc}
  \usepackage[polish]{babel}
```

inputenc z opcją utf8 informuje $\text{\LaTeX}$, że plik źródłowy jest zapisany w kodowaniu UTF-8 — co jest standardem w każdym współczesnym edytorze. fontenc z opcją T1 przełącza zestaw fontów na taki, który zawiera polskie litery jako właściwe glify, a nie złożenia akcentów z literami bazowymi. Bez T1 dzielenie wyrazów dla polskich słów działa źle. babel z opcją polish uaktywnia polskie reguły dzielenia wyrazów i lokalizuje automatyczne napisy jak „Spis treści” czy „Rysunek”.

Nowocześniejsza alternatywa to kompilacja przez Lua $\text{\LaTeX}$ lub Xe $\text{\LaTeX}$ z pakietem polyglossia zamiast babel. Eliminuje potrzebę inputenc (oba kompilatory rozumieją UTF-8 natywnie) i otwiera dostęp do fontów OpenType przez pakiet fontspec. Dla nowych projektów, gdzie zależy ci na nowoczesnej typografii, to lepszy wybór — jednak większość uczelniane szablonów wciąż używa pdflatex z klasycznym zestawem.

Minimalna preambuła dla polskiej pracy dyplomowej
Praktyczna preambuła, z której korzysta większość studentów polskich uczelni, zawiera sześć wierszy: `\documentclass[12pt,a4paper]{report}`, następnie `fontenc` z `T1`, `inputenc` z `utf8`, `babel` z `polish`, `geometry` z marginesami uczelni i `graphicx` dla ilustracji. To minimum, które rozwiązuje wszystkie typowe problemy ze składem polskiego tekstu przed napisaniem pierwszego zdania.

Cztery pakiety na start `geometry` — marginesy. `fancyhdr` — nagłówki i stopki. `graphicx` — grafika. `fontenc` + `inputenc` + `babel` — polskie znaki i dzielenie wyrazów. Bez tych pakietów możesz pisać po angielsku z domyślnym układem strony. Z nimi masz profesjonalny punkt startowy dla każdego polskiego dokumentu.

2.4 Znaki specjalne, cudzysłowy i polskie ogonki: Pułapki, na które wpada każdy początkujący

$\LaTeX$ rezerwuje dziesięć znaków do własnych celów. Wpis ich bezpośrednio w tekście powoduje błąd kompilacji albo — gorzej — cichy błąd typograficzny, którego nie zauważysz bez uważnej lektury wydruku.

Znaki zarezerwowane i ich bezpieczne wersje to pierwsza rzecz, którą warto zapamiętać na pamięć. Znak procenta `%` rozpo-

czyna komentarz — wszystko po nim do końca wiersza jest ignorowane. Dolar \$ przełącza tryb matematyczny. Amper-sand & rozdziela kolumny w tabelach. Tylda ~ wstawia nie-łamliwy odstęp. Daszek ^ i podkreślnik _ oznaczają indeksy górny i dolny w trybie matematycznym. Nawiasy klamrowe { i } grupują argumenty poleceń. Znak # numeruje argumenty w definicjach makr.

Każdy z tych znaków możesz wstawić do tekstu, poprzedzając go odwrotnym ukośnikiem: \%, \&, \#, _, \{, \}. Sam odwrotny ukośnik jest wyjątkiem — \\ to polecenie łamania wiersza, a nie literalny backslash. Do wstawienia odwrotnego ukośnika jako znaku tekstu używasz \textbackslash.

Cudzysłowy to osobna kategoria pułapek. Klawiatura daje ci prosty apostrof i prosty cudzysłów, które $\LaTeX$ traktuje jako znaki tekstowe bez ozdób. Polski cudzysłów dolny otwierający to dwa przecinki wsteczne (, ,), a zamykający — dwa apostrofy ('). Zapis , ,tekst'' daje poprawny wynik typograficzny: „tekst”. Używanie klawiszowego znaku cudzysłowu " zamiast tej konwencji to jeden z najczęstszych błędów w pracach dyplomowych — , ,tekst'' w źródle daje w PDF albo dwa apostrofy skierowane w tę samą stronę, albo znak zupełnie innego języka, zależnie od załadowanego pakietu fontów.

Pauzy rządzą się równie precyzyjnymi regułami. Pojedynczy myślnik - to łącznik w wyrazach złożonych, jak „polsko-angielski”. Dwa myślniki -- dają półpauzę (en-dash), używaną w zakresach liczbowych: „strony 24 – 31”. Trzy myślniki --- to pauza

(em-dash) — dokładnie ta, której używam w tej książce do wtrąceń i pauz narracyjnych. Pomylenie długości myślnika to błąd typograficzny widoczny dla każdego redaktora, choć niewidoczny dla większości czytelników.

Tylda ~ w źródle $\LaTeX$ -a nie wstawia znaku tyldy do tekstu — tworzy niełaŃliwy odstęp. Używasz jej wszędy tam, gdzie złamanie wiersza byłoby typograficznie niedopuszczalne: między liczbą a jednostką (42~km), między inicjałem a nazwiskiem (J.~Kowalski), między skrótem a rozwinięciem (dr~hab.) oraz przed odsyłaczem (Rysunek~\ref{fig:schemat}). Brak niełaŃliwego odstępu to kolejny błąd, który widać dopiero na wydruku, gdy „rys.” łąduje na końcu wiersza, a „1” zaczyna następny.

Cztery błędy kompilacji, które spotkasz w pierwszym tygodniu **Undefined control sequence** — użyłeś polecenia bez wymaganego pakietu w preambule. **Missing \$ inserted** — użyłeś znaku podkreślnika lub daszka poza trybem matematycznym. **Runaway argument** — zapomniałeś zamknąć nawias kłamrowy. **File not found** — ścieżka do pliku graficznego jest błędna lub zawiera spacje. Każdy z tych błędów ma jednoznaczny komunikat w pliku .log i numer wiersza, w którym wystąpił.

Wielokropek to kolejne nieoczekiwane pole minowe. Trzy wpisane po sobie kropki dają w $\LaTeX$ -u odstępy jak po zdaniu — zbyt duże. Poprawne polecenie to \ldots, które wstawia wielokropek z właściwymi odstępami typograficznymi.

Sprawdź cudzysłowy przed wydrukiem Wyszukaj w pliku `.tex` znak `"` (prosty cudzysłów klawiszowy). Każde jego wystąpienie w tekście to prawdopodobnie błąd — zamień na `,`, `,` i `'`. W Overleaf możesz użyć *Find & Replace* z opcją wyrażeń regularnych, by znaleźć wszystkie wystąpienia naraz.

Składnia $\LaTeX$ -a jest precyzyjna — i to zaleta Dziesięć znaków zarezerwowanych, trzy długości myślnika, dwa rodzaje cudzysłowów i niełamliwy odstęp — to cały zestaw reguł, które musisz znać, by uniknąć błędów typograficznych w polskim tekście. Każda z tych reguł ma logiczne uzasadnienie i każdą można zapamiętać przy pierwszym błędzie. System nie jest kapryśny — jest konsekwentny, co oznacza, że gdy raz zrozumiesz regułę, działa zawsze tak samo.

Rozdział 3.

Tekst, który wygląda profesjonalnie: Formatowanie, listy, tabele i grafika

ZNASZ już strukturę dokumentu i umiesz go skompilować. Teraz przychodzi pytanie, które zadaje każdy, kto po raz pierwszy widzi gotowy PDF: skąd $\text{\LaTeX}$ wie, gdzie wstawić numer sekcji, jak wyrównać kolumny tabeli i czemu rysunek znalazł się na górze strony, a nie tam, gdzie go wpisałeś? Odpowiedź na każde z tych pytań jest taka sama — system działa według reguł, które wystarczy zrozumieć raz.

3.1 Hierarchia tekstu: Sekcje, podsekcje, spis treści i automatyczna numeracja

Każdy długi dokument można zniszczyć jedną czynnością: ręczną numeracją rozdziałów. Dopisujesz rozdział trzeci, potem okazuje się, że drugi był zbędny, usuwasz go i nagle wszystkie odwołania w tekście w stylu „patrz rozdział 4” są

błędne. W $\LaTeX$ -u ten problem po prostu nie istnieje.

Hierarchia sekcji w klasie `article` wygląda następująco: `\section{}` tworzy sekcję pierwszego poziomu, `\subsection{}` — drugiego, `\subsubsection{}` — trzeciego. Klasy `report` i `book` dodają jeszcze `\chapter{}` jako poziom nadrzędny. Numeracja jest automatyczna i spójna: jeśli wstawisz nową sekcję między dotychczasowymi drugą a trzecią, wszystkie numery przesuną się bez żadnej interwencji z twojej strony.

```
\section{Wprowadzenie}
\subsection{Cel pracy}
\subsubsection{Zakres badań}
```

Spis treści generujesz jedną linią: `\tableofcontents`. $\LaTeX$ zbiera tytuły i numery stron przy każdej kompilacji, zapisując je w pliku `.toc`. To właśnie dlatego nowy spis treści wymaga dwóch przebiegów kompilacji — przy pierwszym dane są zapisywane, przy drugim odczytywane i wstawiane. Overleaf robi to automatycznie; przy pracy lokalnej możesz użyć `latexmk`, który sam ocenia, ile przebiegów potrzeba.

Gwiazdkowa wersja komendy — `\section*{}` — tworzy sekcję bez numeru i bez wpisu w spisie treści. Przyda się do przedmowy, podziękowań lub aneksu, który ma tytuł, ale nie powinien być częścią struktury numerycznej.

Etykiety do sekcji nadawaj od razu. Tuż po każdym `\section{}` dopisz `\label{sec:nazwa}`. Gdy później będziesz chciał odwo-

łać się do tej sekcji poleceniem `\ref{sec:nazwa}`, $\LaTeX$ wstawi poprawny numer automatycznie. Konwencja z prefiksem `sec:` pochodzi od Karwowskiego i Kaźmierczaka — stosują ją konsekwentnie wszyscy, którzy piszą coś dłuższego niż trzy strony.

Automatyczna numeracja eliminuje całą klasę błędów W typowym dokumencie Word o długości 80 stron zmiana kolejności rozdziałów generuje średnio kilkanaście błędnych odesłań, których autorzy nie zauważają przed drukiem. $\LaTeX$ sprawia, że ta kategoria błędów staje się niemożliwa — numery sekcji, stron i równań to wyniki obliczeń, nie wpisane ręcznie cyfry.

3.2 Wyróżnienia, listy i środowiska: `itemize`, `enumerate`, `description` i bloki cytatów

Różnica między `\textit{tekst}` a `\emph{tekst}` jest subtelna, ale ważna. `\textit{}` bezwarunkowo składa tekst kursywą. `\emph{}` wyróżnia tekst relatywnie do otoczenia: jeśli użyjesz go w normalnym tekście, dostaniesz kursywę; jeśli użyjesz go wewnątrz bloku, który sam jest już kursywą, dostaniesz tekst prosty. To drugie zachowanie jest typograficznie poprawne — wyróżnienie ma kontrastować z otoczeniem.

Tekst normalny z `\emph{wyróżnieniem}`.

`\textit{Blok kursywą z \emph{kontra-wyróżnieniem}.}`

Do pogrubienia służy `\textbf{}`. Używaj go oszczędnie — strona, na której wszystko jest pogrubione, nie wyróżnia niczego. Podkreślenie `\underline{}` jest w typografii książkowej pozostałością po maszynie do pisania i w $\text{\LaTeX}$ -u praktycznie się go nie stosuje.

Listy nieuporządkowane tworzy środowisko `itemize`:

```
\begin{itemize}
  \item Pierwsza pozycja
  \item Druga pozycja
  \item Trzecia pozycja
\end{itemize}
```

Listy numerowane obsługuje `enumerate`. Domyślnie numeruje cyframi arabskimi, ale pakiet `enumitem` — jeden z najczęściej używanych pakietów pomocniczych — pozwala zmienić styl na litery, cyfry rzymskie lub dowolny własny symbol.

Listy opisowe z `description` są przydatne wtedy, gdy każda pozycja ma etykietę i objaśnienie — na przykład słownik pojęć albo zestawienie parametrów:

```
\begin{description}
  \item[preambuła] Część dokumentu przed \begin{document}
  \item[otoczenie] Para ... definiująca blok
\end{description}
```

Środowisko `quote` wstawia blok z obustronnym wcięciem — standardowy sposób na cytaty:

```
\begin{quote}
```

Zbyt przykre było dla mnie oglądanie ostatecznego wyniku.

```
\end{quote}
```

Środowisko `verbatim` wyłącza interpretację znaków specjalnych i składa tekst krojem o stałej szerokości. Wszystkie fragmenty kodu w tej książce korzystają właśnie z niego — lub z jego inlinowej wersji `\verb`.

Nie zagnieżdżaj `verbatim` wewnątrz argumentów Środowisko `verbatim` nie może pojawić się jako argument innego polecenia ani wewnątrz wielu środowisk, w tym `figure`. Jeśli potrzebujesz kodu w podpisie lub w pudełku, użyj pakietu `fancyvrb` albo `listings`, które działają poprawnie w takich kontekstach.

`\emph` kontra `\textit` — zasada kontekstu `\emph{}` to polecenie semantyczne: mówi „wyróżnij to”, a $\TeX$ sam dobiera środek. `\textit{}` to polecenie wizualne: mówi „zrób kursywę”. Pisząc pracę naukową, używaj `\emph{}` wszędzie tam, gdzie zależy ci na znaczeniu, a nie tylko wyglądzie. Gdy w przyszłości zmienisz szablon, wyróżnienia dostosują się automatycznie.

3.3 Tabele bez bólu głowy: Środowisko tabular, pozycjonowanie i pakiet booktabs

Tabela w $\text{\LaTeX}$ -u składa się z dwóch warstw: środowisko `table` (pływające, z podpisem i etykieta) oraz środowisko `tabular` (właściwa siatka danych). Można używać `tabular` bez `table`, ale wtedy tabela jest elementem liniowym bez numeracji.

Argument środowiska `tabular` definiuje kolumny. Litera `l` oznacza wyrównanie do lewej, `c` — wyśrodkowanie, `r` — do prawej. Pionowa kreska `|` wstawia linię między kolumnami. Separator kolumn to `&`, koniec wiersza to `\\`.

```
\begin{tabular}{l c r}
Lewo & Środek & Prawo \\
tekst & tekst & tekst \end{tabular}
```

Polecenie `\hline` rysuje poziomą linię przez całą szerokość tabeli. Domyślne tabele $\text{\LaTeX}$ -a z gęstymi liniami pionowymi i poziomymi wyglądają amatorsko — to pozostałość po sposobie, w jaki tabele składano w latach osiemdziesiątych.

Pakiet `booktabs` rozwiązuje ten problem. Wprowadza trzy polecenia: `\toprule` (gruba linia na górze), `\midrule` (cieńsza linia między nagłówkiem a danymi) i `\bottomrule` (gruba linia na dole). Żadnych linii pionowych, żadnego `\hline`. Efekt jest zbliżony do tabel w renomowanych czasopismach naukowych.

```
\begin{table}[ht]
\centering
\caption{Porównanie klas dokumentów}
\begin{tabular}{llc}
```

```

\toprule
Klasa & Zastosowanie & Rozdziały \\
\midrule
article & artykuły, raporty & nie \\
report & prace dyplomowe & tak \\
book & książki & tak \\
\bottomrule
\end{tabular}
\label{tab:klasy}
\end{table}

```

Tabela 3.1: Porównanie środowisk list w $\LaTeX$ -u

Środowisko	Zastosowanie	Numeracja
itemize	Lista punktowana, kolejność	nie
enumerate	Lista kroków, procedury, wyliczenia	tak
description	Słowniki, zestawienia parametrów	etykiety
quote	Cytaty, wyróżnione bloki tekstu	nie

Podpis tabeli umieszcza się *nad* tabelą — odwrotnie niż w przypadku rysunków, gdzie `\caption{}` należy wstawić pod grafiką. To konwencja typograficzna, a nie wymóg techniczny, ale jej nieprzestrzeganie jest zauważalne dla każdego recenzenta.

Jak tabele z booktabs wyglądają w praktyce naukowej Journal of the American Statistical Association oraz IEEE Transactions on Information Theory wymagają od autorów tabel bez pionowych linii i z separatorami `\toprule`/`\midrule`/`\bottomrule`. Szablony tych czasopism dostępne na CTAN używają booktabs domyślnie. Autorzy przesyłający pracę w formacie `.tex` z tabelami opartymi na `\hline` otrzymują standardowy komentarz recenzenta: „proszę użyć booktabs”.

3.4 Grafika i wstawki: `includegraphics`, `figure`, pozycjonowanie i podpisy

$\LaTeX$ nie wstawia grafiki „w miejscu” — robi to przeglądarka internetowa. Zamiast tego traktuje rysunki jako *wstawki* (ang. *floats*): prostokątne bloki, które system sam lokuje w pobliżu miejsca, gdzie zostały umieszczone w kodzie, ale z uwzględnieniem estetyki strony.

Mechanizm jest prosty: $\LaTeX$ szuka pierwszego miejsca, które spełnia kryteria pozycjonowania, i wstawia tam rysunek. Kryteria definiuje opcjonalny argument środowiska `figure`:

- `h` — tutaj (here), jeśli jest miejsce
- `t` — na górze strony (top)
- `b` — na dole strony (bottom)
- `p` — na osobnej stronie z wstawkami
- `!` — zignoruj wewnętrzne limity liczby wstawek

Kombinacja [htbp] oznacza: spróbuj najpierw tutaj, potem na górze, potem na dole, potem na osobnej stronie. To rozsądny domyślny wybór dla większości dokumentów.

Pakiet `graphicx` obsługuje wstawianie plików graficznych. Przy kompilacji `pdflatex` obsługiwane formaty to PDF, PNG i JPEG. Dla grafiki wektorowej (wykresy, schematy) zdecydowanie najlepszym wyborem jest PDF — zachowuje ostrość przy dowolnym powiększeniu.

```
\begin{figure}[htbp]
  \centering
  \includegraphics[width=0.8\textwidth]{schemat.pdf}
  \caption{Schemat połączeń w sieci lokalnej}
  \label{fig:schemat}
\end{figure}
```

Kluczowa zasada: `\label{}` musi znaleźć się *po* `\caption{}`, nie przed. Etykieta zawsze odnosi się do ostatniego numerowanego bytu w danym zakresie — jeśli wstawisz ją przed podpisem, `\ref{}` zwróci numer sekcji zamiast numeru rysunku.

Nie używaj spacji w nazwach plików graficznych $\LaTeX$ interpretuje spację jako separator argumentów. Plik `mój schemat.pdf` spowoduje błąd kompilacji. Stosuj podkreślniki (`mój_schemat.pdf`) lub myślniki (`mój-schemat.pdf`). To samo dotyczy polskich znaków w ścieżkach — bezpieczniej ich unikać.

Float to nie błąd — to funkcja. Gdy rysunek „ucieknie” na następną stronę, wielu początkujących traktuje to jako usterkę i próbuje to naprawić siłą, dodając `[h!]` lub wstawiając ręczne łamanie stron. To droga donikąd. $\LaTeX$ umieścił rysunek tam, gdzie skład wygląda lepiej. Zaakceptuj tę decyzję — albo użyj pakietu `float` z opcją `[H]`, który wymusi dosłowną pozycję, świadomie rezygnując z estetyki składu.

3.5 Matematyka w praktyce: Od wzoru inline do równań numerowanych z pakietem `amsmath`

Fizyka, matematyka i informatyka mają w $\LaTeX$ -u swojego naturalnego sojusznika — i nie jest to przypadek. Donald Knuth zaprojektował $\TeX$ -a właśnie dlatego, że standardowy skład komputerowy lat siedemdziesiąt

ych nie nadawał się do złożenia wzoru całkowego. Efekt tej frustracji to system, który do dziś nie ma konkurencji w składzie matematyki.

$\LaTeX$ oferuje trzy tryby matematyczne. Tryb inline wstawia wzór w linii tekstu — między znakami dolara: `$E = mc^2$` daje $E = mc^2$. Tryb display wyróżnia wzór w osobnej przestrzeni, wyśrodkowany i z większymi odstępami:

```
\[
\int_0^{\infty} e^{-x^2} \, dx = \frac{\sqrt{\pi}}{2}
```

\]

Trzeci tryb to równanie numerowane — środowisko `equation`. Numer pojawia się automatycznie po prawej stronie i można się do niego odwołać przez `\ref{}` lub `\eqref{}` (pakiet `amsmath` dodaje nawiasy okrągłe wokół numeru, co jest standardem w publikacjach naukowych):

```
\begin{equation}
  F = G \frac{m_1 m_2}{r^2}
  \label{eq:newton}
\end{equation}
```

Równanie~`\eqref{eq:newton}` opisuje prawo grawitacji.

Podstawowe elementy składni matematycznej warto znać na pamięć: indeks dolny to `_`{}, górny to `^`{}. Ułamek to `\frac{licznik}{mianownik}`. Pierwiastek to `\sqrt{}` lub `\sqrt[n]{}` dla pierwiastka n -tego stopnia. Suma to `\sum_{i=1}^n`, całka to `\int_a^b`.

Pakiet `amsmath`, stworzony przez American Mathematical Society, jest de facto obowiązkowy przy każdym dokumencie z równaniami. Dodaje środowisko `align`, które pozwala wyrównać układ równań względem wybranego znaku — zwykle znaku równości:

```
\begin{align}
  a &= b + c \\
  d &= e - f + g
\end{align}
```

Każda linia dostaje własny numer. Jeśli nie chcesz numerować konkretnej linii, dopisz `\nonumber` przed `\\`. Jeśli nie chcesz numerować całego środowiska, użyj wersji gwiazdkowej `align*`.

Tabela 3.2: Najczęściej używane polecenia matematyczne w $\text{\LaTeX}$ -u

Polecenie	Znaczenie	Przykład
<code>^{\{}}</code>	Indeks górny (potęga)	x^{2n}
<code>_{\{}}</code>	Indeks dolny	a_{ij}
<code>\frac {\{ }\{ }</code>	Ułamek	$\frac{p}{q}$
<code>\sqrt {\{ }}</code>	Pierwiastek kwadratowy	$\sqrt{2}$
<code>\sum</code>	Symbol sumy	$\sum_{k=1}^n$
<code>\int</code>	Symbol całki	$\int_0^1$
<code>\infty</code>	Nieskończoność	∞

Dlaczego $\text{\LaTeX}$ jest standardem w naukach ścisłych? Liczby mówią same za siebie: ponad 90% artykułów w *Physical Review Letters* jest składanych w $\text{\LaTeX}$ -u, a arXiv.org — serwer preprintów z ponad 2 milionami artykułów — przyjmuje pliki `.tex` jako format podstawowy i sam kompiluje je do PDF. Żaden inny system składu nie oferuje porównywalnej jakości wzorów przy zachowaniu czytelnego kodu źródłowego.

arXiv i standard $\text{\LaTeX}$ -a w fizyce Od 1991 roku arXiv.org zebrał ponad 2,3 miliona artykułów naukowych. Ponad 70% z nich zostało przesłanych jako pliki $\text{\LaTeX}$ -owe, a nie PDF — co oznacza, że autorzy udostępniają kod źródłowy, nie tylko wynik

kompilacji. To umożliwia wydawnictwom takim jak Springer czy Elsevier bezpośrednie przejęcie pliku `.tex` i dostosowanie go do szablonu czasopisma bez ponownego przepisywania treści. Word nie oferuje niczego porównywalnego.

Jeden błąd, który popełnia każdy początkujący: pisanie wzorów bez trybu matematycznego. Litera x wpisana poza dolarem jest składana jako litera tekstowa, bez właściwego pochylenia i odstępów. $\LaTeX$ nie ostrzeże cię przed tym błędem — po prostu wynik będzie wyglądał źle. Wzory, zmienne i symbole matematyczne zawsze umieszczaj między `...` lub `\(...\)`.

Nie używaj `$$...$$` w $\LaTeX$ -u Podwójny dolar `$$...$$` pochodzi z Plain $\TeX$ -a i w $\LaTeX$ -u zachowuje się nieprzewidywalnie — może zaburzać numerację równań i odstępy pionowe. Zamiast tego używaj `\[...\]` dla wzorów wyróżnionych bez numeru i środowiska `equation` dla wzorów numerowanych. To nie tylko kwestia stylu — to kwestia poprawności kompilacji.

Matematyka w $\LaTeX$ -u to inwestycja jednorazowa Nauczenie się składni wzorów zajmuje kilka godzin. Efekt trwa przez całą karierę naukową. Wzory napisane w $\LaTeX$ -u są przenośne, przeszukiwalne, zrozumiałe dla innych badaczy i niezależne od wersji oprogramowania. Artykuł złożony w $\LaTeX$ -u w 1995 roku skompiluje się bez zmian dziś — czego nie można powiedzieć o plikach `.doc` z tej samej epoki.

Trzy rozdziały za tobą i masz już kompletny warsztat: wiesz,

jak zbudować dokument, jak go skompilować i jak nadać mu profesjonalny wygląd. Brakuje jednego elementu, bez którego żadna praca naukowa nie jest kompletna — bibliografii, skrowidzy i własnych szablonów, które przekształcają wiedzę o $\text{\LaTeX}$ -u w gotowe narzędzie pracy.

Od pliku do gotowej pracy: Bibliografia, skorowidze i własne szablony

PRACA dyplomowa bez bibliografii to nie praca dyplomowa — to szkic. A bibliografia tworzona ręcznie, pozycja po pozycji, z ręcznie pilnowaną numeracją, to przepis na błędy, które wychodzą na jaw dopiero po wydruku. $\text{\LaTeX}$ rozwiązuje ten problem systemowo, za pomocą narzędzia, które istnieje od 1985 roku i nie wymaga od ciebie zapamiętania ani jednej reguły formatowania cytowań.

4.1 Bibliografia bez ręcznego formatowania: BibTeX, plik .bib i style cytowań

Oren Patashnik i Leslie Lamport stworzyli jako odpowiedź na jeden konkretny problem: każde czasopismo naukowe wymaga innego formatu cytowań, a przepisywanie bibliografii przy każdym złożeniu artykułu jest stratą czasu. Rozwiąza-

nie jest eleganckie — dane bibliograficzne przechowujesz raz, w zewnętrznym pliku `.bib`, a styl wybierasz osobno, jednym poleceniem.

Plik `.bib` to zwykły plik tekstowy. Każda pozycja zaczyna się od deklaracji typu — `@article` dla artykułów w czasopiśmie, `@book` dla książek, `@mastersthesis` dla prac magisterskich. Po typie następuje unikalny klucz, którym odwołujesz się do pozycji w tekście. Przykładowy wpis artykułu wygląda tak:

```
@article{kowalski2019,  
  author = ,,Jan Kowalski and Anna Nowak'',  
  title = ,,Analiza metod optymalizacji'',  
  journal = ,,Informatyka Teoretyczna'',  
  volume = ,,12'',  
  number = ,,3'',  
  pages = ,,45 -- 67'',  
  year = ,,2019''  
}
```

Zwróć uwagę na dwie rzeczy: autorów oddzielasz słowem `and` (nie przecinkiem), a zakres stron zapisujesz z podwójnym myślnikiem (`45 --- 67`), co w $\text{\LaTeX}$ -u daje półpauzę. Wpis książki różni się zestawem pól — zamiast `journal`, `volume` i `pages` podajesz `publisher` i `address`:

```
@book{goossens93,  
  author = "Michel Goossens and Frank Mittelbach  
  and Alexander Samarin",
```



```
title = ,,The LaTeX Companion'',  
publisher = ,,Addison-Wesley'',  
address = ,,Reading, Massachusetts'',  
year = ,,1993''  
}
```

W tekście głównym wstawiasz cytowanie poleceniem `\cite{kowalski201}` a $\LaTeX$ automatycznie numeruje pozycje i dopasowuje nawiasy do wybranego stylu. Na końcu dokumentu dodasz dwa polecenia:

```
\bibliographystyle{plain}  
\bibliography{moja_baza}
```

Pierwsze wybiera styl formatowania, drugie wskazuje plik `moja_baza.bib`. Styl `plain` sortuje pozycje alfabetycznie i numeruje je — to domyślny wybór dla prac technicznych. Styl `alpha` używa skrótów literowych zamiast liczb, na przykład `[Goo93]` dla pozycji Goossensa z 1993 roku. Styl `apalike` stosuje format autor – rok, popularny w naukach społecznych i humanistycznych. Dla polskich prac Marek Gagolewski rekomenduje `plplain` — odpowiednik `plain` ze spolonizowanymi nagłówkami.

Kompilacja z `-em` wymaga czterech kroków: `pdflatex`, `bibtex`, `pdflatex`, `pdflatex`. Pierwsze uruchomienie generuje plik pomocniczy `.aux` z listą kluczy, `bibtex` buduje na jego podstawie plik `.bbl` z sformatowaną bibliografią, a dwa kolejne uruchomienia `pdflatex` wstawiają numery cytowań do tekstu.

Tabela 4.1: Porównanie najczęściej stosowanych stylów bibliografii w -u

Styl	Format cytowania	Zastosowanie
plain	Numeryczny [1], sortowanie alfabetyczne	Prace techniczne, informatyka
alpha	Skrót literowy [Goo93]	Matematyka, logika
apalike	Autor – rok (Kowalski, 2019)	Nauki społeczne, psychologia
plplain	Numeryczny, polskie nagłówki	Polskie prace dyplomowe
unsrt	Numeryczny, kolejność cytowania	Artykuły konferencyjne

Overleaf wykonuje te kroki automatycznie.

Jeden plik .bib na całą karierę Plik .bib możesz podłączyć do dowolnej liczby dokumentów. Badacze, którzy przez lata zbierają pozycje w jednym pliku, mogą błyskawicznie złożyć bibliografię każdego nowego artykułu. Narzędzia takie jak JabRef (darmowy) lub Zotero (z eksportem .bib) pozwalają zarządzać bazą graficznie, bez ręcznej edycji pliku tekstowego.

4.1.1 Biber i BibLaTeX: nowoczesna alternatywa, której wymaga coraz więcej uczelni

Podczas gdy BibTeX przez dekady był standardem, dziś wiele polskich uczelni oraz międzynarodowych wydawnictw (m.in. Elsevier i Springer w swoich nowych szablonach) wymaga użycia pakietu **BibLaTeX** wraz z silnikiem **Biber**. To tandem, który zastępuje klasyczny BibTeX i oferuje znacznie większą elastyczność.

Podstawowa różnica polega na tym, że BibLaTeX przejmuje kontrolę nad formatowaniem bibliografii bezpośrednio w LaTeX-u, a Biber zastępuje program bibtex jako procesor pliku .bib. Dzięki temu możliwe jest pełne wsparcie dla Unicode (w tym polskich znaków w nazwiskach autorów), zaawansowane style cytowań oraz łatwa zmiana stylu bez modyfikacji pliku źródłowego.

Aby skorzystać z BibLaTeX, wystarczy dodać do preambuły:

```
[language=TeX]
```

literatura.bib

Następnie w miejscu, gdzie ma pojawić się bibliografia, wstawiamy:

```
[language=TeX]
```

Kompilacja wymaga teraz trzech kroków: `pdflatex` → `biber` → `pdflatex` (dwukrotnie). W Overleaf dzieje się to automatycznie

po wybraniu Biber jako domyślnego procesora w ustawieniach projektu.

Który styl wybrać? Najpopularniejsze style BibLaTeX to `numeric` (numeryczne odniesienia w nawiasach kwadratowych), `authoryear` (styl harwardzki) oraz `verbose-ibid` (stosowany w humanistyce). Styl zmieniasz jedną opcją w `\usepackage`, bez dotykania reszty dokumentu.

Nie mieszaj BibTeX z Biberem. Jeśli w preambule masz `backend=biber`, a kompilujesz klasycznym `bibtex`, bibliografia nie zostanie wygenerowana i nie pojawi się żaden czytelny komunikat o błędzie. Sprawdź ustawienia kompilatora przed wysłaniem pracy.

4.1.2 Zarządzanie dużymi projektami: `\input`, `\include` i automatyzacja z `latexmk`

Kiedy dokument przekracza kilkadziesiąt stron — na przykład praca dyplomowa, książka czy obszerny raport techniczny — trzymanie całości w jednym pliku `.tex` staje się niepraktyczne. LaTeX oferuje dwa polecenia do podziału projektu na mniejsze pliki: `\input{plik}` oraz `\include{plik}`.

`\input` wstawia zawartość wskazanego pliku dokładnie w miejscu wywołania, bez żadnych efektów ubocznych — można go używać wielokrotnie i zagnieźdzać. `\include` działa podobnie, ale automatycznie dodaje `\clearpage` przed i po wstawieniu, co jest wygodne przy rozdziałach. Dodatkowo współpracuje z

poleceniem `\includeonly{rozdział2,rozdział3}` w preambule, które pozwala kompilować tylko wybrane pliki, zachowując przy tym poprawną numerację stron i odsyłaczy.

Zalecana struktura katalogów dla większego projektu wygląda następująco:

```
praca/  
  main.tex           \% plik główny  
  rozdzialy/  
    wstep.tex  
    metodologia.tex  
    wyniki.tex  
  grafiki/  
    wykres1.pdf  
  bibliografia.bib
```

Do automatyzacji kompilacji służy narzędzie `latexmk`, które samodzielnie wykrywa, ile razy należy uruchomić `pdflatex`, `bibtex` czy `makeindex`, aby wszystkie odsyłacze i spis treści były aktualne. Wystarczy jedno polecenie:

```
latexmk -pdf main.tex
```

Tryb ciągłej kompilacji Opcja `latexmk -pdf -pvc main.tex` uruchamia *podgląd na żywo*: narzędzie obserwuje zmiany w plikach i automatycznie rekompiluje dokument po każdym zapisie. To ogromna oszczędność czasu podczas intensywnej pracy nad tekstem.

latexmk jest dostępny w każdej standardowej dystrybucji TeX Live i MiKTeX, więc nie wymaga osobnej instalacji.

4.2 Odsyłacze, skorowidze i przypisy: `\label`, `\ref`, `\footnote` i `makeindex`

Numerowanie rozdziałów, rysunków i tabel w $\text{\LaTeX}$ -u jest automatyczne — ale żeby *odwołać się* do konkretnego elementu, potrzebujesz etykiety. System działa na zasadzie klucz – wartość: przypisujesz etykietę i przywołujesz ją w dowolnym miejscu tekstu.

Etykietę dodajesz poleceniem `\label{klucz}` bezpośrednio po nagłówku sekcji, po `\caption{}` rysunku lub tabeli, albo po środowisku `equation`. Potem odwołujesz się do numeru przez `\ref{klucz}`, a do strony — przez `\pageref{klucz}`. W polskich tekstach akademickich obowiązuje konwencja opisana przez Gagoleskiego: *Rozdział 1* na początku zdania, *rozdz. 1* w środku. Tylda między słowem a numerem zapobiega łamaniu wiersza w tym miejscu — pomiń ją, a $\text{\LaTeX}$ może wstawić numer na początku następnego wiersza.

Konwencja nazewnictwa etykiet Przyjmij schemat `typ:nazwa`, na przykład `sec:wstep`, `fig:schemat`, `tab:wyniki`, `eq:rownanie`. Gdy dokument urośnie do stu stron, odróżnienie etykiety sekcji od etykiety rysunku w kodzie źródłowym oszczędza wiele czasu.

Przypisy dolne wstawiasz poleceniem `\footnote{tekst przypisu}` bezpośrednio w miejscu, gdzie ma pojawić się znacznik. $\LaTeX$ automatycznie numeruje przypisy i umieszcza je na właściwej stronie — nie musisz pilnować, czy numer przypisu zgadza się z treścią na dole. To pozornie drobna funkcja, ale przy pracy magisterskiej z kilkudziesięcioma przypisami wyeliminowanie ręcznej numeracji jest odczuwalną oszczędnością.

Skorowidz — indeks rzeczowy na końcu książki lub obszernego raportu — generujesz w trzech krokach. Najpierw ładujesz pakiet `makeidx` i w preambule wywołujesz `\makeindex`. Następnie w tekście oznaczasz terminy poleceniem `\index{hasło}`. Dla podhasła używasz składni `\index{hasło!podhasło}`. Po kompilacji uruchamiasz program `makeindex nazwa.idx`, który sortuje wpisy i tworzy plik `.ind`. Ostatecznie wstawiasz gotowy indeks poleceniem `\printindex`.

Dwukrotna kompilacja po zmianach w indeksie Po każdej zmianie oznaczeń `\index{}` musisz ponownie uruchomić `makeindex`, a potem `pdflatex`. Jeśli skompilujesz tylko `pdflatex`, indeks będzie nieaktualny, ale $\LaTeX$ nie ostrzeże cię o tym błędzie.

Etykiety i indeks — architektura nawigacji dokumentu System `\label`–`\ref` i `makeindex` to dwa poziomy tej samej filozofii: $\LaTeX$ zarządza numeracją, ty zarządzasz znaczeniem. Zmień kolejność rozdziałów, dodaj nowy rysunek na początku — wszystkie numery zaktualizują się same. Ręczna numeracja w edytorze

WYSIWYG wymagałaby przejrzenia całego dokumentu.

4.3 Własne komendy i pakiety: Jak przestać kopiować kod i zacząć myśleć jak programista

Każdy, kto pisze pracę z dziedziny, gdzie pewne oznaczenia powtarzają się dziesiątki razy, w pewnym momencie zaczyna kopiować te same fragmenty kodu. To znak, że nadszedł czas na własne komendy.

Polecenie `\newcommand` przyjmuje dwa argumenty: nazwę nowej komendy i jej definicję. Jeśli w całej pracy stosujesz notację R^n dla przestrzeni euklidesowej, zamiast pisać `\mathbb{R}^n` za każdym razem, możesz zdefiniować skrót:

```
\newcommand{\Rn}{\mathbb{R}^n}
```

Od tej chwili `\Rn` daje R^n wszędzie. Komendy mogą też przyjmować argumenty — na przykład komenda do wektora:

```
\newcommand{\wektor}[1]{\mathbf{#1}}
```

Teraz `\wektor{v}` składa się jako $\mathbf{v}$. Symbol `#1` oznacza pierwszy argument. Możesz zdefiniować do dziewięciu argumentów.

Własne środowiska tworzy się przez `\newenvironment`. Definicja zawiera kod otwierający i zamykający środowisko:


```
\newenvironment{twierdzenie}
{\textbf{Twierdzenie.} \itshape}
{}
```

Gdy kilkanaście takich definicji zaczyna wypełniać preambułę, czas na własny plik `.sty`. To zwykły plik tekstowy z rozszerzeniem `.sty`, zaczynający się od `\ProvidesPackage{nazwa}`. Wrzucasz do niego wszystkie `\newcommand`, `\newenvironment` i `\usepackage`, które chcesz stosować w wielu dokumentach. Potem ładujesz go jednym poleceniem: `\usepackage{nazwa}`. Jeden plik `.sty` może obsługiwać całą serię artykułów, prac dyplomowych i raportów — zmiana stylu cytowań albo kroju pisma wymaga modyfikacji w jednym miejscu, nie w każdym dokumencie osobno.

Jak Leslie Lamport używał własnych makr Lamport pisząc oryginalną dokumentację $\text{\LaTeX}$ -a stosował własne makra do formatowania przykładów kodu i komend. Każde polecenie opisywane w podręczniku miało dedykowane środowisko, które automatycznie formatowało je z odpowiednim krojem i dodawało do indeksu. Zmieniając definicję jednego środowiska, aktualizował wygląd setek przykładów jednocześnie. Ta sama zasada obowiązuje w każdej pracy, gdzie ten sam typ elementu powtarza się więcej niż pięć razy.

`\newcommand` to nie wygoda — to utrzymywalność. Własne komendy nie są luksusem dla zaawansowanych. Są odpowiedzią na pytanie: czy za rok, gdy wrócisz do tego dokumentu, będziesz pamiętał, co oznacza każdy fragment kodu? Komenda

`\wektor{v}` jest czytelna. `\mathbf{v}` wymaga znajomości kontekstu.

4.4 Co dalej? Mapa zasobów: CTAN, GUST, Overleaf Learn i TeX Stack Exchange

Żaden podręcznik nie wyczerpuje $\text{\LaTeX}$ -a. System liczy ponad 6 000 pakietów w archiwum CTAN (*Comprehensive TeX Archive Network*) — każdy rozwiązuje konkretny problem, od składu nut muzycznych po generowanie kodów QR. Znać wszystkich pakietów nie musisz — musisz wiedzieć, gdzie szukać.

CTAN (ctan.org) to centralne repozytorium. Każdy pakiet ma stronę z dokumentacją w formacie PDF, listą zależności i datą ostatniej aktualizacji. Szukając pakietu do konkretnego zadania, wpisz temat w wyszukiwarkę CTAN — na przykład *bibliography*, *tables* albo *multilingual*. Wyniki zawierają krótkie opisy i linki do pełnej dokumentacji.

Polska Grupa Użytkowników Systemu $\text{\TeX}$ — GUST — prowadzi portal gust.org.pl z zasobami skierowanymi bezpośrednio do polskiego czytelnika. Znajdziesz tam polskie tłumaczenie klasycznego podręcznika *Nie za krótkie wprowadzenie do systemu $\text{\LaTeX}2\epsilon$* autorstwa Oetikera, Partla, Hyny i Schlegl — to wciąż najlepszy punkt wejścia dla osoby, która chce przejść od podstaw do średniozaawansowanego poziomu w jednym dokumencie liczącym nieco ponad 80 stron. GUST wydaje też

Biuletyn GUST — recenzowane pismo poświęcone $\text{\TeX}$ -owi, dostępne bezpłatnie w archiwach serwisu.

Dla pytań szczegółowych — i jest ich nieuchronnie wiele — najskuteczniejszym narzędziem jest *TeX Stack Exchange* (tex.stackexchange.com). Serwis liczy ponad 250 000 pytań i odpowiedzi; większość problemów, które napotkasz jako początkujący, ktoś już tam opisał i rozwiązał. Zadając własne pytanie, dołącz *minimalny przykład kompilujący się samodzielnie* (ang. *Minimal Working Example*, MWE) — fragment kodu, który reprodukuje problem w możliwie krótkim dokumencie. Społeczność odpowiada szybciej i trafniej, gdy widzi konkretny kod zamiast opisu słownego.

Overleaf Learn (overleaf.com/learn) to platforma edukacyjna z kilkuset artykułami pokrywającymi tematy od absolutnych podstaw po zaawansowane techniki składu. Artykuły są utrzymywane na bieżąco — w odróżnieniu od wielu blogów, które opisują $\text{\LaTeX}$ -a w wersjach sprzed dekady.

Jak szybko znaleźć pakiet do konkretnego zadania Wpisz opis problemu w angielski Google z frazą `latex package`, na przykład `latex package professional tables`. Pierwsze wyniki zwykle prowadzą albo do TeX Stack Exchange, albo do dokumentacji CTAN. Nie instaluj pakietu, zanim nie przeczytasz pierwszych dwóch stron jego dokumentacji — większość błędów kompilacji u początkujących wynika z pominięcia jednej wymaganej opcji ładowania.

4.5 Dokąd stąd: Podsumowanie całej drogi

Cztery rozdziały tej książki prowadziły przez jedną myśl: $\LaTeX$ nie jest trudny — jest *inny*. W rozdziale pierwszym zobaczyłeś, dlaczego Knuth zaprojektował $\TeX$ -a jako system, który separuje treść od formy. W rozdziale drugim zbudowałeś pierwszy działający dokument i zrozumiałeś, czym różni się klasa `article` od `book`. Rozdział trzeci dał ci narzędzia do profesjonalnego składu tekstu, tabel i grafiki. Ten rozdział domknął obraz: automatyczna bibliografia, etykiety i odsyłacze, własne komendy — to elementy, które odróżniają dokument złożony w $\LaTeX$ -u od dokumentu *napisanego* w $\LaTeX$ -u.

Teraz wiesz, jak złożyć kompletną pracę dyplomową. Baza `.bib` zarządza cytowaniami, `\label` i `\ref` pilnują numeracji, `\newcommand` eliminuje powtórzenia, a CTAN czeka z rozwiązaniem każdego problemu, na który jeszcze nie trafiłeś.

Jedna umiejętność, która procentuje przez całą karierę Opanowanie $\LaTeX$ -a to inwestycja, która zwraca się każdym kolejnym dokumentem. Artykuł złożony dziś skompiluje się bez zmian za dwadzieścia lat — bo $\TeX$ jest stabilny z zasady, nie z przypadku. Zaczynj od jednego prostego dokumentu. Dodawaj po jednym narzędziu. Za miesiąc $\LaTeX$ przestanie być nowym językiem, a stanie się twoim naturalnym sposobem pisania.